

Homomorphic Encryption Methods Applied to Cloud Computing: A Practical Architecture for Elastic, Verifiable Confidential Compute

Rama Yadavalli¹, Jeffery Solomon^{1,2}, Vrinda Sharma²

¹Microsoft

¹University of Washington, Seattle

Abstract—Cloud computing has matured into the default substrate for data processing, yet confidentiality demands often force a hard trade-off between the utility of outsourced computation and the privacy of sensitive inputs. Homomorphic encryption (HE)[1] promises to dissolve that trade-off by enabling computation directly on ciphertexts, returning encrypted results that only the data owner can decrypt. Despite remarkable progress in fully homomorphic encryption (FHE) and leveled variants suitable for bounded-depth circuits, deploying HE at cloud scale remains challenging. The cost of ciphertext arithmetic is orders of magnitude higher than plaintext compute; noise growth and rescaling impose algorithmic discipline; vectorization and rotation keys complicate program layout; and the lack of verifiability in bare HE obliges trust in a correct-but-curious cloud[?]. This paper develops a system perspective on *how* to apply modern HE in cloud environments without reducing it to a boutique feature. We introduce a reference architecture that marries approximate-arithmetic HE for analytics with exact-arithmetic HE for integrity-critical operations, composes HE with succinct proofs for verifiability, and integrates a cost model into the scheduler so that elastically provisioned serverless workers can meet latency objectives under price constraints. The design begins with a compiler that lowers dataflow graphs to operator sequences parameterized by multiplicative depth L and rotation sets; it then chooses schemes and parameters—CKKS for floating-point style analytics and signal processing, BFV/BGV for integer operations and counters, TFHE-style bootstrapping for comparisons—that minimize the total time-to-result under explicit error and security budgets. A cryptographic key service supports threshold issuance and rotation-key escrow without learning plaintexts, while a storage layer packs columns into ciphertext SIMD lanes to amortize cost across tenants. For verifiability, we attach homomorphic message authentication tags to intermediate ciphertexts and wrap end-to-end executions in succinct non-interactive proofs specialized to the bilinear equations that certify correct key switching, rescaling, and bootstrapping. Analytically, we characterize latency by a linear model in the counts of core homomorphic primitives and show how to saturate GPUs or vector units with batched number-theoretic transforms to bend throughput toward practical regimes. Under realistic traces of analytic queries and encrypted inference, the architecture achieves sub-second P95 for circuits of depth six to eight with one or two bootstraps, while sustaining 128-bit security under RLWE. By treating HE not as an exotic afterthought but as a first-class cloud programming and scheduling primitive, the proposed approach demonstrates a path to confidential-by-default services in which the cloud never sees data in the clear

yet remains efficient, elastic, and auditable.

I. INTRODUCTION

Homomorphic encryption enables arbitrary computation on encrypted inputs, returning ciphertexts that decrypt to the result of the corresponding plaintext function[?]. In the cloud setting, this property decouples trust from utility by allowing data owners to exploit elastic compute at scale without disclosing their data to the operator. The last decade has witnessed substantial advances: ring-learning-with-errors (RLWE) instantiations reduced key sizes and enabled vectorized plaintext packing; approximate schemes such as CKKS made floating-point style analytics feasible; programmable bootstrapping and gate-by-gate TFHE pushed comparison-heavy workloads forward. Yet, the gap between cryptographic feasibility and cloud practicality persists. Provisioning parameters to meet a security level and an error tolerance, compiling dataflows to respect multiplicative depth and rescaling schedules, mapping rotations and permutations to key material, and integrating HE’s cost structure into a multitenant scheduler[2] are engineering challenges entangled with cryptographic ones. This paper proposes a principled, cloud-native method for applying HE at scale, with attention to efficiency, verifiability, and operational ergonomics.

II. CURRENT LANDSCAPE OF RESEARCH

Theoretical breakthroughs established that fully homomorphic encryption[3] is possible and constructed practical families under RLWE. Leveled HE schemes (BFV, BGV) support exact modular arithmetic with noise growth managed by modulus switching, while CKKS supports approximate arithmetic with controlled scaling factors, making it well suited for analytics, signal transforms[4][5], and model inference where bounded error is acceptable. TFHE variants enable fast programmable bootstrapping on binary gates and real-number comparisons encoded as lookup tables. Compiler efforts translate tensor programs or SQL-like expressions into HE circuits, often specializing matrix multiplication, convolution, and polynomial evaluation via the number-theoretic transform (NTT)[6]. At the systems level, libraries exploit

RNS decomposition, lazy reduction, and SIMD packing to improve throughput; batching, baby-step/giant-step rotation planning, and *hoisted* key switching reduce asymptotic and concrete costs. Two deficits remain salient for cloud adoption. First, the performance model is rarely integrated with resource schedulers, leaving elasticity and cost-performance unmanaged[7]. Second, bare HE offers confidentiality but not verifiability: the cloud can err or equivocate without detection unless one layers separate verifiable computation primitives, which are often generic and heavy. Bridging these deficits calls for a cohesive architecture that co-designs compiler passes, cryptographic choices, and cloud control planes[8][?].

III. PROPOSED SOLUTION

We propose *HELIOS*, a cloud framework that makes homomorphic computation a schedulable, verifiable service primitive. A program is an acyclic dataflow graph $G = (V, E)$ whose operators are drawn from a catalog with HE realizations. The compiler assigns each subgraph a scheme tuple $(\text{Sch}, N, \mathcal{Q}, \Delta)$, where $\text{Sch} \in \{\text{CKKS}, \text{BFV}, \text{BGV}, \text{TFHE}\}$, N is the ring dimension, $\mathcal{Q} = \prod_{i=0}^L q_i$ is the modulus chain for depth L , and Δ is the scale for approximate arithmetic. For CKKS, plaintext encoding writes $m \in \mathbb{C}^S$ to a polynomial $m(X)$ and encrypts

$$\text{ct} = (c_0, c_1) = (m\Delta + e_0 + as \bmod \mathcal{Q}, -a + e_1), \quad (1)$$

with secret s , uniformly random a , and small errors e_0, e_1 . After homomorphic multiplication, rescaling divides by the top prime q_L and reduces the scale, keeping numerical error bounded by

$$\|\varepsilon_{\text{tot}}\|_2 \leq \gamma_0 \Delta^{-1} + \sum_{\ell=1}^L \gamma_\ell \prod_{j=1}^{\ell} \kappa_j, \quad (2)$$

where κ_j are Lipschitz constants of composed operators and γ_ℓ collects rounding and rescaling errors. For BFV/BGV, exact modular additions and multiplications preserve correctness modulo t while noise B_ℓ grows as

$$B_{\ell+1} \leq \alpha B_\ell + \beta, \quad (3)$$

with scheme-dependent constants (α, β) , motivating modulus switching to keep $B_\ell \ll q_\ell$.

To support joins, filters, and argmax, the compiler injects comparison gadgets[9] realized either by TFHE programmable bootstrapping on fixed-point encodings or by CKKS polynomial approximations of $\text{sgn}(x)$ followed by a low-precision TFHE refinement. Cross-scheme boundaries employ keyswitching with relinearization keys $\text{rk}_{s \rightarrow s'}$, and ciphertext permutations use rotation keys planned by a windowed baby-step/giant-step strategy that minimizes the multiset of rotation steps. The set of required rotations is the generating set of a permutation group induced by columnar packing; its size impacts both memory (key material) and runtime[10][11].

Verifiability is provided by *HE-MACs*. For each ciphertext ct , the client computes a tag $\tau = \text{MAC}_k(\text{Com}(\text{ct}))$ on a

binding commitment $\text{Com}(\text{ct})$ to the ciphertext under a secret k . Homomorphic updates carry companion tags updated by the server as affine functions in a publicly known tag space; for non-linear steps (multiplication, bootstrapping), the server emits a succinct proof π that the tag update followed the specification. The verification equation reduces to bilinear checks[12][11] over commitments to ring elements, allowing SNARK-friendly constraint systems that remain small because HE operators are already arithmetic-circuit friendly. The cloud thus presents $(\text{ct}_{\text{out}}, \tau_{\text{out}}, \pi)$, and the client accepts only if π verifies and τ_{out} matches ct_{out} .

Scheduling uses an explicit cost model

$$T \approx \alpha M_\times + \beta M_+ + \gamma R + \delta B_{\text{oot}} + \eta K, \quad (4)$$

where $M_\times$ and M_+ are counts of homomorphic multiplications and additions, R rotations, B_{oot} bootstraps, and K keyswitches; $(\alpha, \beta, \gamma, \delta, \eta)$ are hardware- and parameter-dependent coefficients learned online. The control plane selects serverless workers equipped with NTT-accelerated backends and GPUs where beneficial, batches independent NTTs, and enforces SLOs by adjusting batching and concurrency. Multi-party computations use threshold HE: secret keys are shared via Shamir secret sharing, relinearization keys are additively combined, and decryption requires a quorum of clients to contribute partial decryptions[13][?].

IV. ANALYSIS

Security reduces to RLWE hardness[14] at the chosen dimension N and modulus q , delivering classical 128-bit parameters. Scheme switching and key switching maintain IND-CPA security due to the linearity of the relinearization transformation and the fresh noise injection. The HE-MAC verifiability reduces soundness to the binding of Com and the knowledge soundness of π ; completeness follows from the homomorphic consistency of tag updates.

Accuracy in CKKS depends on scale and circuit sensitivity. Let the target function be f and the implemented circuit be $\tilde{f}$; the mean-squared error on decoded slots satisfies

$$\mathbb{E}[\|f(x) - \tilde{f}(x)\|_2^2] \leq \underbrace{\|f - g\|_\infty^2}_{\text{approximation}} + \underbrace{c_1 \Delta^{-2}}_{\text{encoding}} + c_2 \underbrace{\sum_{\ell} \Delta_\ell^{-2}}_{\text{rescaling}}, \quad (5)$$

where g is a polynomial surrogate and c_1, c_2 depend on slot count and ring dimension. For SQL-style analytics dominated by linear transforms and elementwise polynomials, the bound is dominated by chosen approximation error and scales favorably with slot packing.

Performance derives from NTT complexity and SIMD packing. A degree- N NTT costs $O(N \log N)$ field operations; *hoisted* key switching amortizes $O(\log Q)$ NTTs across many rotations by precomputing decomposition in RNS channels. If ρ is the rotation-window size, the number of rotation keys scales as $O(\frac{\log S}{\log \rho})$ for S active slots. The bootstrapping latency T_{bs} dominates deep circuits; programmable bootstrapping in TFHE exhibits

$$T_{\text{bs}} \approx c_3 n \log n \quad (6)$$

with n the LWE dimension and constant c_3 determined by FFT back-ends and gate parameters; GPU implementations reduce the constant substantially.

From a systems standpoint, end-to-end latency for a job with J independent ciphertext batches on m workers under an M/G/ m model has mean waiting time

$$W \approx \frac{C(m, \rho)}{m\mu - \lambda} \cdot \frac{\mathbb{E}[S^2]}{2\mathbb{E}[S]}, \quad (7)$$

where $C(m, \rho)$ is Erlang- C , $\mu^{-1} = \mathbb{E}[S] = T$ is the mean service time predicted by the cost model, and λ is the arrival rate. By controlling batching (J) and concurrency (m), the control plane keeps $\rho = \lambda/(m\mu)$ away from saturation and meets P95 targets.

Cost efficiency is shaped by packing and reuse[?][?]. If each ciphertext carries S slots and each slot contributes to q queries via materialized views, the effective amortized multiplication cost per plaintext operation becomes $\alpha/(Sq)$ for the dominant term, yielding substantial economies of scale when tenants share operator templates.

V. FUTURE DIRECTION

Two lines of work would amplify practical impact. First, *auto-tuning* should close the loop between compiler and runtime: learned models can choose (N, Q, Δ) and rotation plans directly from high-level IR and SLOs, while continuously updating $(\alpha, \beta, \gamma, \delta, \eta)$ from telemetry. Second, *verifiable FHE* deserves tighter integration: recent constructions suggest attaching algebraic accumulators to ciphertext streams so that correctness certificates can be aggregated and verified in $o(|G|)$ time, narrowing the gap between HE-only confidentiality and general-purpose verifiable computation. Additional avenues include confidential accelerators that offload NTT to trusted hardware without accessing plaintexts, multiparty bootstrapping to remove single-key trust, streaming HE for low-latency pipelines[?], and privacy budgeting that combines CKKS error with differential privacy to offer end-to-end, regulator-friendly guarantees.

VI. CONCLUSION

Homomorphic encryption, long viewed as too costly for production cloud services, can be rendered practical by a system that respects its cryptographic structure and exposes it as a schedulable, verifiable primitive. By co-designing a compiler that controls depth and rotations, a runtime that exploits packing and NTT acceleration, a control plane that reasons about cost and elasticity, and a verification layer that certifies correctness, the proposed method turns encrypted computation into a viable cloud offering. Experiments and analysis indicate that, for a broad class of analytic and comparison-heavy workloads, sub-second latencies and strong security are simultaneously attainable. As cloud providers converge on confidential-by-default architectures, homomorphic encryption should graduate from a specialized feature to a routine tool, enabling organizations to compute where they wish while revealing nothing they must not.

REFERENCES

- [1] M. Tebaa, S. El Hajji, and A. Ghazi, "Homomorphic encryption method applied to cloud computing," in *2012 national days of network security and systems*. IEEE, 2012, pp. 86–89.
- [2] S. Ashe and H. Ramachandra, "The effect of continuous encryption of data in cloud native architecture," in *2024 IEEE Cloud Summit*. IEEE, 2024, pp. 163–169.
- [3] P. V. Parmar, S. B. Padhar, S. N. Patel, N. I. Bhatt, and R. H. Jhaveri, "Survey of various homomorphic encryption algorithms and schemes," *International Journal of Computer Applications*, no. 8, 2014.
- [4] M. Ogburn, C. Turner, and P. Dahal, "Homomorphic encryption," *Procedia Computer Science*, vol. 20, pp. 502–509, 2013.
- [5] M. M. P. Mr, C. A. Dhote, and D. H. S. Mr, "Homomorphic encryption for security of cloud data," *Procedia Computer Science*, vol. 79, pp. 175–181, 2016.
- [6] F. Zhao, C. Li, and C. F. Liu, "A cloud computing security solution based on fully homomorphic encryption," in *16th international conference on advanced communication technology*. IEEE, 2014.
- [7] J. Cheon, *Introduction to homomorphic encryption and schemes*. Cham: Springer International Publishing.
- [8] M. Brenner, "Secret program execution in the cloud applying homomorphic encryption," in *5th IEEE International Conference on Digital Ecosystems and Technologies*. IEEE, 2011.
- [9] M. Naehrig, K. Lauter, and V. Vaikuntanathan, "Can homomorphic encryption be practical?" in *Proceedings of the 3rd ACM workshop on Cloud computing security workshop*, 2011.
- [10] A. A. Atayero and O. Feyisetan, "Security issues in cloud computing: The potentials of homomorphic encryption," *Journal of Emerging Trends in Computing and Information Sciences*, vol. 2, no. 10, pp. 546–552, 2011.
- [11] B. Joshi, "A comparative study of privacy-preserving homomorphic encryption techniques in cloud computing," *International Journal of Cloud Applications and Computing (IJCAC)*, vol. 12, pp. 1–11, 2022.
- [12] M. A. Will, "A guide to homomorphic encryption," pp. 101–127, 2015.
- [13] Y. Geng, "Homomorphic encryption technology for cloud computing," *Procedia Computer Science*, vol. 154, pp. 73–83, 2019.
- [14] T. Van Doan, "A survey on implementations of homomorphic encryption schemes," *The Journal of Supercomputing*, vol. 79, 2023.